LIGO GNSS Processing: Implementation Modifications

Engineering Change Summary

1 August 2026

1 Scope

This document summarizes the modifications made to the LIGO GNSS processing pipeline. The work had two objectives: improve the structure and maintainability of the original monolithic GNSS processing implementation, add a base-station/rover carrier-phase processing path that produces a floating-point RTK solution through double-difference factors, and resolve validated carrier ambiguities to integers for a fixed RTK solution.

2 GNSS processing redesign

The large processing flow in `src/GNSS_Processing_fg.cpp` was separated into focused operations with explicit responsibilities. The extracted logic includes:

- GNSS buffer clearing and initialization-window shifting;
- receiver clock prediction;
- insertion of frame state estimates into GTSAM;
- extraction of optimized states from the iSAM2 estimate;
- carrier-phase history pruning by frame number and observation time;
- LiDAR information-matrix normalization; and
- selection of the first valid constellation clock bias during GNSS initialization.

Repeated buffer-management and state-copying blocks were replaced by these helpers. Factor insertion order and factor identifiers were retained so that the existing fixed-lag graph-deletion behavior remains compatible. LiDAR information normalization now detects a non-positive diagonal before scaling, preventing division by zero and marking the LiDAR update invalid for that epoch.

3 Base-station observation input

The new `BaseStationData` component reads surveyed base-station coordinates and carrier observations from RINEX 3.x observation data, with RINEX 3.02 verified explicitly. Its input may be a plain observation file, a ZIP archive containing an observation member, or a directory of files and archives. ZIP members are read in place through `libzip`; no manual extraction or CSV conversion is required. The base antenna position is read in Earth-Centered, Earth-Fixed (ECEF) coordinates from:

```
APPROX POSITION XYZ
```

The reader assembles multiline `SYS / ## / OBS TYPES` header records, reads L1/L5 carrier phase and LLI fields, applies GLONASS frequency channels, deduplicates multiple tracking codes on one satellite/frequency, and handles special RINEX event records. GPS and Galileo bands 1 and 5, BeiDou B1I (RINEX 3.02 band 1, with band 2 also accepted) and B2a, and GLONASS L1 are mapped to their physical carrier frequencies. UTC and BDT observation times are converted to GPST before epochs are stored and matched to the rover time within a configurable tolerance.

RINEX LLI and SSI fields do not encode carrier standard deviation, so the reader assigns the configured `base_carrier_std_cycles` value. It rejects malformed headers, absent ECEF coordinates, inconsistent positions across files, and inputs with no supported carrier observations. The active configuration points to `Data/2026-Aug-01_233602`, which contains two consecutive zipped HKSC observation files. Their actual observation epoch is 21 March 2025; the enclosing directory date is only the local data-directory name and must not be used for synchronization.

4 Double-difference carrier factors

For every synchronized rover/base epoch, the processing pipeline performs the following operations:

1. Match rover and base observations by satellite and carrier frequency.
2. Retain the primary band and, when enabled, GPS L5, Galileo E5a, and BeiDou B2a observations at 1176.45 MHz.
3. Group observations by GNSS constellation and signal band.
4. Select the lowest-noise carrier observation in each constellation/band group as its reference satellite.
5. Form the carrier measurement double difference

$$\Delta\nabla L = (L_r^s - L_b^s) - (L_r^q - L_b^q),$$

   where r and b denote rover and base receivers, s is the subject satellite, and q is the same-constellation reference satellite.
6. Add a geometric range double-difference factor and a persistent, real-valued ambiguity state to the iSAM2 graph.

Two factor variants were implemented in `include/gnss_factor/gnss_double_diff_carrier_factor.hpp`. The first uses the local LiDAR/IMU position together with the estimated local-to-ECEF alignment. The second directly uses the ECEF navigation state when LiDAR is disabled. Both factors include analytical Jacobians for the connected pose, position, alignment, and ambiguity variables.

The ambiguity associated with a satellite/reference/band tuple is initialized from the difference between measured and predicted geometry. It remains a continuous GTSAM `Vector1` state across epochs. A new ambiguity is created after a receiver or base loss-of-lock indication, an observation gap, or a change of reference pair. L1 and L5 therefore have independent ambiguity arcs and wavelengths. Until integer validation succeeds, these states provide the floating-point RTK solution.

5 LAMBDA integer ambiguity resolution

The new `LambdaAmbiguityResolver` implements integer least-squares decorrelation and search. Ambiguities and their full joint marginal covariance are converted from metres to cycles before resolution. Only same-frequency double differences are eligible; this excludes combinations whose metre-valued ambiguity cannot be represented by one integer wavelength. All eligible primary-band and L5 ambiguities enter the same joint covariance and integer search; no ionosphere-free carrier combination is used because that would destroy the direct integer nature of the original ambiguities.

Integer fixing is guarded by a minimum number of ambiguities, a continuous lock-epoch requirement, a maximum float standard deviation, and the ratio of the second-best to best LAMBDA squared residual. Partial ambiguity resolution removes the highest-variance ambiguity and retries when the complete set is not sufficiently precise or does not pass validation. Failure at any gate leaves the float graph unchanged.

After a successful ratio test, each accepted integer is converted back to metres and inserted as a tight ambiguity prior. iSAM2 is updated a second time in the same epoch, so the constrained navigation state is returned immediately. The constraint is held for the continuous ambiguity arc; a receiver/base loss-of-lock flag or observation gap starts a new float ambiguity instead of reusing the old integer.

6 Configuration

The following parameters were added to the GNSS section of `config/avia.yaml`:

Parameter	Default	Purpose
`use_double_differences`	false	Enable base/rover carrier factors
`use_l5`	true	Add supported 1176.45 MHz carrier factors
`base_station_file`	HKSC directory	RINEX file, ZIP, or directory under `Data`
`base_epoch_tolerance`	0.05 s	Maximum rover/base epoch offset
`base_carrier_std_cycles`	0.01 cycles	Assumed RINEX phase uncertainty
`double_difference_sigma`	0.02 m	Carrier factor standard deviation
`ambiguity_prior_sigma`	100 m	Loose float-ambiguity prior
`enable_integer_fixing`	true	Enable validated LAMBDA fixing
`lambda_min_ambiguities`	4	Smallest partial-fix subset
`lambda_min_lock_epochs`	10	Required continuous ambiguity arc
`lambda_ratio_threshold`	3.0	Integer candidate ratio gate
`lambda_max_std_cycles`	0.25 cycles	Float precision gate
`fixed_ambiguity_sigma_cycles`	0.001 cycles	Fix-and-hold prior sigma

Double differences remain disabled by default because the supplied base data must be paired with a rover bag covering the same 21 March 2025 GPST epochs. For a synchronized run, set `use_double_differences` to `true`.

7 Build and verification

The GNSS processor header and implementation now contain structured source annotations for the class lifecycle, graph-key convention, initialization branches, fixed-lag update, state import/export, satellite preparation, six factor-assembly stages, L1/L5 ambiguity arcs, and LAMBDA validation. The comments describe ownership, frames, and failure behavior rather than merely restating individual C++ expressions. A separate pipeline reference is provided in

`paper/GNSS_Processing_fg_pipeline.tex`. The LAMBDA resolver is likewise annotated with its matrix convention, unimodular transform, integer Gauss operations, conditional-variance permutations, depth-first zig-zag enumeration, pruning radius, candidate ordering, back-transformation, and float-fallback failure behavior.

The build definition now compiles `src/BaseStationData.cpp` and `src/LambdaAmbiguityResolver.cpp` into `ligo_mapping`. The base reader and its test executable link `libzip`. The package-level CMake configuration no longer overwrites a caller-supplied build type, which also resolves a mismatch between the package and catkin's bundled GoogleTest library.

The complete ROS executable was built and linked successfully with a single-job Release build. Base-file parsing remains in `test/test_base_station_data.cpp`. A separate RTK core suite in `test/test_rtk_core.cpp` verifies both double-difference factor variants, including zero residuals and every analytical Jacobian against central finite differences. It also verifies a known correlated LAMBDA solution, candidate ordering, integer-translation invariance, invalid-input rejection, and 40 deterministic covariance cases against exhaustive integer search.

The RTK suite also verifies primary/L5 classification, frequency-specific base matching, disabling L5, and rejection of unsupported GLONASS L5. The base reader suite verifies a compact multiline RINEX 3.02 fixture and directly loads the two actual HKSC ZIP archives, including L1/L5 extraction, surveyed ECEF coordinates, aggregation across both hours, epoch matching, and a flag-4 receiver event. The two LIGO test executables reported ten passing tests. The complete catkin result collector, including the workspace's generated test records, reported:

20 tests, 0 errors, 0 failures, 0 skipped.

Runtime accuracy and convergence of the float/fixed solution still require a rover bag whose observations and ephemerides overlap the base RINEX GPST epochs.